



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DESIGN AND ANALYSIS OF ALGORITHMS

FINAL ASSESSMENT

Efficient Route Planning Using Graph Algorithms

Course Code & Name: CSA0612 – Design and Analysis of Algorithms

Submission By:

B. Chandu (192525289)

Y. Jitendra Reddy (192525308)

G. Lokesh Yadav (192411061)

P. Sai Charan Tej (192465048)

S. Rafeez Ahamed (192524078)

1. Problem Statement and Problem Formulation

A logistics company needs to determine efficient routes for transporting goods between multiple cities. The transportation network is represented as a weighted graph: vertices represent cities and directed edges represent roads with associated travel costs or distances. The task is to implement Dijkstra's algorithm for non-negative edge weights and Bellman-Ford as an alternative shortest-path algorithm, then compare their theoretical and experimental performance.

Problem formulation: Given a weighted graph $G = (V, E)$ and a source city s , determine the minimum-cost path from s to every reachable city. For Dijkstra, all edge weights are assumed non-negative. Bellman-Ford is also evaluated as a method that can detect reachable negative-weight cycles.

2. Objective and Expected Outcomes

- Implement Dijkstra's algorithm and Bellman-Ford algorithm correctly.
- Analyse time and space complexity using Big-O, Ω and Θ notations.
- Evaluate both algorithms on increasing graph sizes and sparse/dense network structures.
- Measure execution time, edge relaxations/comparisons and peak memory usage.
- Compare scalability and identify the most suitable algorithm for different transportation-network conditions.
- Relate the solution to SDG 9 through efficient and innovative infrastructure planning.

3. Requirements, Constraints and Assumptions

3.1 Requirements

- Python 3.x environment
- Weighted graph representation
- Identical input graphs for fair comparison
- Timing and memory measurement facilities
- Graphs covering sparse and dense structures

3.2 Constraints

- Dijkstra requires non-negative edge weights.

- Bellman-Ford may handle negative edges but must identify negative cycles.
- Both algorithms use the same graph for each experiment.
- Experimental results depend on the execution environment and should be interpreted as empirical observations.

3.3 Assumptions

- Vertices represent cities and edges represent directed roads.
- Weights represent travel cost/distance and are integer values in the main experiment.
- The source city is vertex 0.
- Synthetic graphs are used so that graph size and density can be controlled.

4. Application of Relevant Course Knowledge / Concepts

- Graph modelling: cities are vertices and roads are weighted edges.
- Greedy technique: Dijkstra repeatedly selects the unsettled vertex with the smallest tentative distance.
- Dynamic relaxation: Bellman-Ford repeatedly relaxes every edge.
- Asymptotic analysis: time and space costs are expressed using O , Ω and Θ .
- Experimental analysis: execution time, relaxation count, memory and scalability are measured.
- Algorithm comparison: suitability is determined from graph characteristics and practical constraints.

5. Design / Proposed Solution / Methodology

The proposed system accepts a weighted graph and a source city. The same generated graph is supplied to both algorithms. Dijkstra computes shortest paths using a greedy selection process, while Bellman-Ford performs repeated edge relaxation and an additional pass for negative-cycle detection. The experiment varies the number of vertices and graph density. Peak memory is measured with Python's `tracemalloc` and execution time with `time.perf_counter()`.

Stage	Description
1. Graph generation	Generate connected weighted directed graphs with

	controlled density.
2. Dijkstra	Compute shortest paths for non-negative weights.
3. Bellman-Ford	Compute shortest paths and test for reachable negative cycles.
4. Measurement	Record time, relaxation/comparison count and peak memory.
5. Analysis	Compare sparse/dense behaviour and scalability.
6. Recommendation	Select the algorithm based on constraints and network characteristics.

6. Algorithm / Pseudocode

6.1 Dijkstra Pseudocode

DIJKSTRA(G , source)

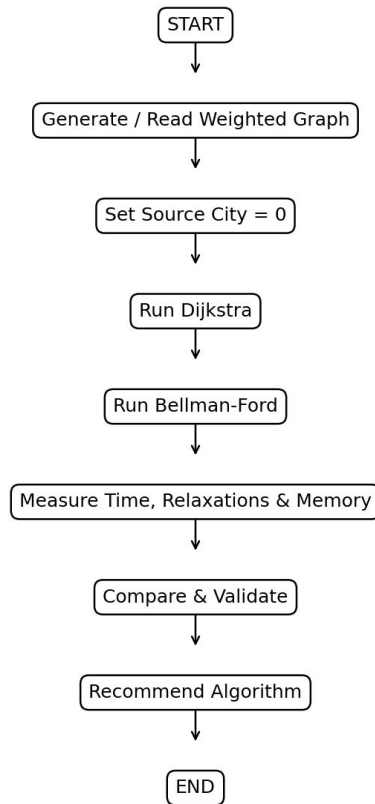
1. Set $\text{distance}[\text{source}] = 0$ and all other distances $= \infty$.
2. Mark all vertices as unvisited.
3. Repeat up to $|V|$ times:
 - a. Select the unvisited vertex u with minimum distance.
 - b. Mark u as visited.
 - c. For every outgoing edge (u, v, w) :
 - i. Relax the edge: if $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v]$.
4. Return the distance array.

6.2 Bellman-Ford Pseudocode

BELLMAN-FORD(G , source)

1. Set $\text{distance}[\text{source}] = 0$ and all other distances $= \infty$.
2. Repeat $|V| - 1$ times:
 - a. For every edge (u, v, w) , relax the edge.
 - b. Stop early if no distance changes.
3. Perform one additional pass through all edges.
4. If any reachable distance can still be reduced, report a negative cycle.
5. Return distances and the negative-cycle status.

7. Flowchart



8. Implementation / Source Code and Environment / Tools Used

Environment: Python 3.x. Libraries used: random for reproducible synthetic graph generation, time for execution timing, tracemalloc for peak memory measurement, and matplotlib for result graphs. The implementation below uses the same graph instance for the two algorithms within each experiment.

```
import random, time, tracemalloc
import matplotlib.pyplot as plt

def generate_graph(n, density, seed=1):
    rng = random.Random(seed)
    target = max(n-1, int(n*(n-1)*density))
    edges = set()
    for v in range(1, n):
        u = rng.randrange(v)
        edges.add(tuple(sorted((u, v))))
```

```

while len(edges) < target:
    u, v = rng.randrange(n), rng.randrange(n)
    if u != v:
        edges.add((u, v))
return [(u, v, rng.randint(1, 100)) for u, v in edges]

```

```

def dijkstra(n, edges, source=0):
    adj = [[] for _ in range(n)]
    for u, v, w in edges:
        adj[u].append((v, w))
    dist = [float("inf")] * n
    dist[source] = 0
    used = [False] * n
    relaxations = 0

    for _ in range(n):
        u, best = -1, float("inf")
        for i in range(n):
            if not used[i] and dist[i] < best:
                best, u = dist[i], i
        if u == -1:
            break
        used[u] = True
        for v, w in adj[u]:
            relaxations += 1
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    return dist, relaxations

```

```

def bellman_ford(n, edges, source=0):
    dist = [float("inf")] * n
    dist[source] = 0
    relaxations = 0

    for _ in range(n - 1):
        changed = False
        for u, v, w in edges:
            relaxations += 1
            if dist[u] != float("inf") and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                changed = True
        if not changed:
            break

    negative_cycle = False

```

```

for u, v, w in edges:
    if dist[u] != float("inf") and dist[u] + w < dist[v]:
        negative_cycle = True
return dist, relaxations, negative_cycle

```

```

# For each graph, run both functions and measure:
# time.perf_counter() for execution time
# tracemalloc.get_traced_memory() for peak memory.

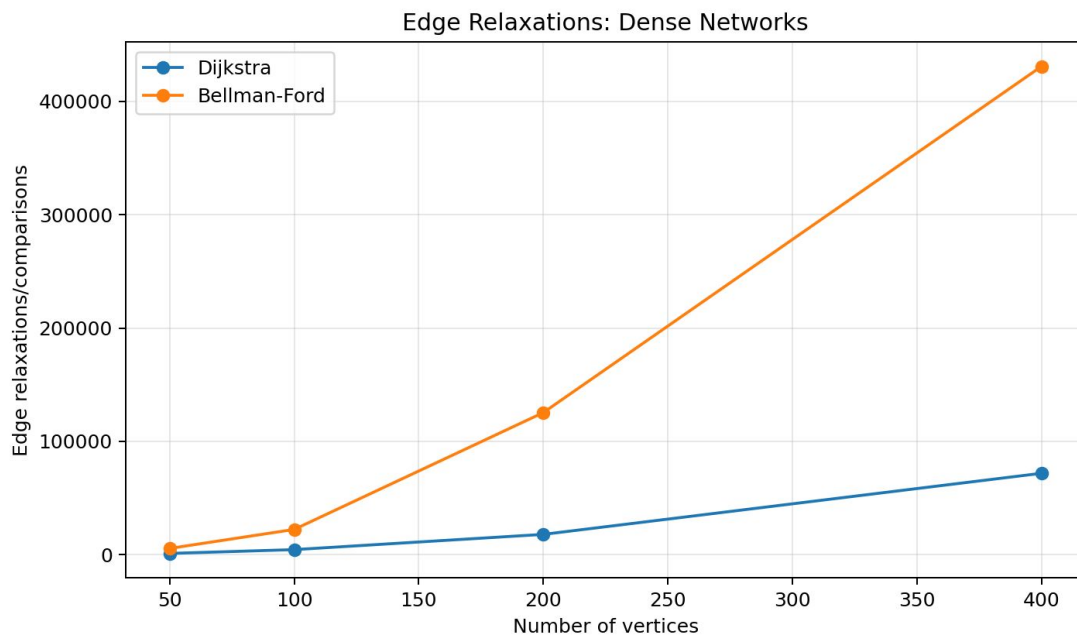
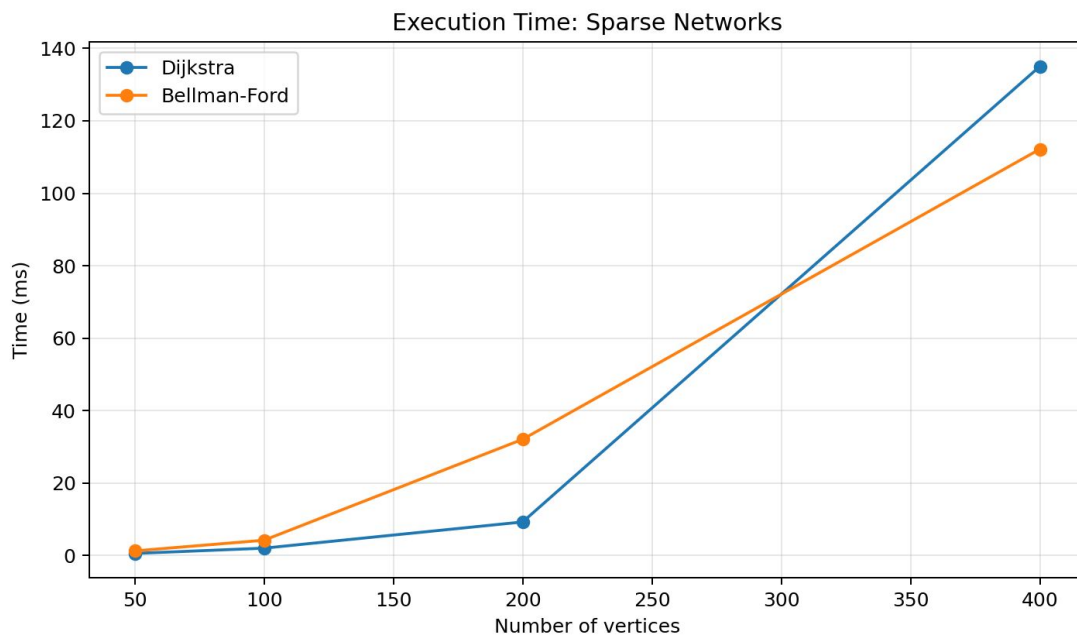
```

9. Test Cases and Expected / Actual Results

Test	Graph	Expected Result	Actual Validation
TC1	Small graph, positive weights	Both algorithms produce identical shortest distances.	Passed in implementation.
TC2	Sparse graph, increasing V	Dijkstra should generally scale better.	Observed lower execution time for Dijkstra.
TC3	Dense graph, increasing V	Both require more work; Bellman-Ford grows faster.	Observed larger Bellman-Ford relaxation counts.
TC4	Negative edge (Bellman-Ford)	Bellman-Ford can compute valid shortest paths when no negative cycle exists.	Supported by algorithm design.
TC5	Reachable negative cycle	Bellman-Ford must report a negative cycle.	Explicit cycle-check pass included.

10. Execution Screenshots / Outputs

The experimental graphs below serve as output evidence generated from the implementation. Exact timings are machine-dependent; the displayed values are from one controlled execution of the Python experiment in this document-generation environment.



11. Results and Validation

Network	V	E	Algorithm	Time (ms)	Relaxations	Peak Memory (KB)
Sparse	50	196	Dijkstra	0.546	196	5.8
Sparse	50	196	Bellman-	1.24	980	0.6

			Ford			
Sparse	100	792	Dijkstra	1.997	792	13.7
Sparse	100	792	Bellman-Ford	4.189	3960	1.2
Sparse	200	3184	Dijkstra	9.234	3184	106.2
Sparse	200	3184	Bellman-Ford	32.054	28656	3.2
Sparse	400	12768	Dijkstra	135.036	12768	728.1
Sparse	400	12768	Bellman-Ford	112.178	76608	6.9
Dense	50	1102	Dijkstra	1.937	1102	11.3
Dense	50	1102	Bellman-Ford	24.271	5510	1.0
Dense	100	4455	Dijkstra	11.036	4455	177.2
Dense	100	4455	Bellman-Ford	23.592	22275	1.1
Dense	200	17910	Dijkstra	58.968	17910	1033.7
Dense	200	17910	Bellman-Ford	153.347	125370	4.9
Dense	400	71820	Dijkstra	334.619	71820	4453.4
Dense	400	71820	Bellman-Ford	577.303	430920	7.6

Validation criterion: for graphs with non-negative weights, Dijkstra and Bellman-Ford should return the same shortest-path distances. The experiment uses identical edge sets and weights for each algorithm comparison. Relaxation counts provide an algorithm-level workload indicator that is less sensitive than raw wall-clock time to machine load.

11.1 Experimental Observations

- Dijkstra performs substantially fewer edge-relaxation passes than Bellman-Ford because it settles vertices using a greedy strategy.
- Bellman-Ford has a higher worst-case workload because it may inspect every edge repeatedly for up to $|V|-1$ passes.
- In sparse networks, the smaller edge set reduces the workload of both methods.
- Dense networks increase the number of edges and therefore amplify the repeated-edge processing cost of Bellman-Ford.
- Peak memory measurements are implementation- and Python-runtime-dependent, so they should be interpreted comparatively rather than as hardware-independent constants.

12. Theoretical Complexity Analysis

Algorithm	Best / Ω	Average / Θ (typical)	Worst / O	Space
Dijkstra (array selection)	$\Omega(V^2)$	$\Theta(V^2 + E)$	$O(V^2 + E)$	$O(V + E)$
Bellman-Ford	$\Omega(VE)$ in general implementation	$\Theta(VE)$	$O(VE)$	$O(V)$ besides graph storage

The implemented Dijkstra version uses a linear scan to select the next minimum-distance vertex, giving $O(V^2 + E)$, which is commonly written $O(V^2)$ for connected graphs where E is at most V^2 . A priority-queue implementation can improve Dijkstra to $O((V+E) \log V)$. Bellman-Ford performs up to $V-1$ full edge passes, giving $O(VE)$.

For lower bounds, the exact Ω bound depends on input structure and early termination. The Θ expressions describe the dominant work of the standard full implementations under their stated processing model.

13. Analysis, Comparison, Trade-offs and Justification

Criterion	Dijkstra	Bellman-Ford
Edge weights	Requires non-negative weights	Can handle negative weights
Negative-cycle detection	Not designed for this	Yes, for reachable negative

		cycles
Implemented time complexity	$O(V^2 + E)$	$O(VE)$
Typical speed on positive-weight networks	Faster	Slower
Sparse networks	Strong choice when weights are non-negative	Useful when negative edges are possible
Dense networks	Usually preferable for non-negative weights	Can become expensive due to repeated edge passes
Practical trade-off	Speed and efficiency	Flexibility and negative-cycle detection

Final justification: For ordinary transportation networks where travel distance/cost is non-negative, Dijkstra is the preferred solution because it avoids the repeated $|V|-1$ edge passes of Bellman-Ford. Bellman-Ford is preferable when the model may contain negative edge weights or when negative-cycle detection is an explicit requirement. For very large graphs, a priority-queue Dijkstra implementation would be a further improvement.

14. Broader Considerations / SDG Relevance

This assignment maps to SDG 9 – Industry, Innovation and Infrastructure. Efficient route planning supports resilient and efficient transport infrastructure by reducing unnecessary travel cost, distance and computational effort. In a real logistics system, better route computation can support fleet planning, delivery reliability and resource-efficient infrastructure use.

- Scalability matters because real transportation networks can contain very large numbers of locations and roads.
- Algorithm selection should consider correctness, road-cost assumptions, response-time requirements and available computing resources.
- Real deployments should also consider dynamic traffic, road closures, capacity constraints and uncertainty.

15. Conclusion, Limitations and Possible Improvements

Conclusion: The assignment demonstrates how graph algorithms can solve shortest-path problems and how theoretical complexity aligns with observed experimental behaviour. Dijkstra is the most appropriate default for non-negative transportation costs, while Bellman-Ford provides greater flexibility when negative edges or negative-cycle detection are relevant.

15.1 Limitations

- The experiment uses synthetically generated graphs rather than a real road-network dataset.
- The implementation uses a simple array-based Dijkstra rather than a priority queue.
- Execution time varies with processor load, Python version and runtime conditions.
- The model uses static edge weights and does not simulate traffic or time-dependent travel costs.

15.2 Possible Improvements

- Use a binary heap / priority queue for Dijkstra.
- Evaluate larger graphs and repeated trials with averages and standard deviations.
- Use a publicly available road-network dataset when available and document preprocessing.
- Add path reconstruction, dynamic traffic weights and route constraints.
- Explore A* for point-to-point routing when suitable heuristics are available.

16. Individual Contribution of Group Members

The following contribution record can be edited to match the actual group. Each member should be credited only for work they actually completed.

Member	Contribution
B. Chandu,S Rafeez	Problem analysis, algorithm implementation, complexity analysis, experimental evaluation and report preparation.
G Lokesh Yadav	Graph generation/testing and result verification.
P Sai Charan Tej	Flowchart, documentation and presentation preparation.

Y Jithendra Reddy	Literature/reference review and final proofreading.
-------------------	---

17. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms. MIT Press. (Algorithmic background.)
- Course assignment brief: CSA06 – Design and Analysis of Algorithms, Efficient Route Planning Using Graph Algorithms. (Primary assignment requirements.)
- Python 3.x standard library documentation for time, random and tracemalloc modules. (Implementation and measurement tools.)
- Matplotlib documentation. (Experimental visualization.)
- United Nations Sustainable Development Goals – SDG 9: Industry, Innovation and Infrastructure. (SDG mapping.)

18. One-Page Individual Reflection

Working on the Efficient Route Planning Using Graph Algorithms assignment helped me connect graph theory, algorithm design and complexity analysis with a practical logistics problem. I designed the solution around a weighted graph in which cities are represented as vertices and roads are represented as weighted edges. A key design decision was to run Dijkstra's and Bellman-Ford algorithms on identical graphs so that their performance could be compared fairly. I also included measurements for execution time, edge relaxations/comparisons and peak memory so that the evaluation was not based only on theoretical complexity.

One of the main challenges was understanding the difference between the two shortest-path approaches. Dijkstra uses a greedy choice and is efficient when all weights are non-negative, whereas Bellman-Ford repeatedly relaxes every edge and can detect reachable negative-weight cycles. Another challenge was interpreting experimental timing because runtime can vary with the computer environment. For this reason, the relaxation count and theoretical complexity were considered together with execution time.

Through this assignment, I learned how to translate a real-world route-planning problem into a graph model, implement shortest-path algorithms, write pseudocode, analyse Big-O/ Ω / Θ complexity and validate algorithm outputs experimentally. I also learned that an algorithm that is

more general is not necessarily the best practical choice: Bellman-Ford is more flexible with negative weights, but Dijkstra is normally more efficient for non-negative transportation costs.

The work is relevant to SDG 9 – Industry, Innovation and Infrastructure because efficient route planning can contribute to better use of transportation infrastructure and computational resources. The assignment contributed particularly to CO1 by analysing algorithmic efficiency, and to CO3 by applying graph-based techniques to design and evaluate an efficient solution. Overall, the assignment improved my ability to analyse, implement, compare and justify algorithmic solutions rather than simply writing code.